

ZenShield OTA Release Implementation and Operations Guide

Version 1.0 | 8 September 2026

For the ZenShield engineering, QA, release engineering, platform, and support teams

1 Purpose and delivery scope

Implement the ZenPlus release update cycle for ZenShield: build one signed release package, publish it through Zentryc, offer it only to eligible ZenShield appliances, apply it safely, verify application and data health, and report the outcome. This guide defines the implementation work, integration contract, test gates, and operating procedure the team needs to deliver that cycle.

ZenShield release management belongs at <https://zentryc.com/ota/zenai/releases/>. ZenPlus remains at <https://zentryc.com/ota/releases/>. Keep the supplied zenai portal route even though the product display name is ZenShield. A portal URL is a management page, not an upload API or a package download location.

Use the ZenPlus Linux appliance architecture as the starting point. The ZenShield paths, account names, service names, and health route below are proposed implementation conventions. Confirm them against the ZenShield application before coding. Adapt runtime and database handlers to the actual ZenShield stack; PostgreSQL, ClickHouse, Python, Go, and Vite are ZenPlus components, not assumed ZenShield dependencies.

Required outcome: a genuine prior-version ZenShield appliance upgrades through the production offer mechanism, preserves its identity and data, passes functional checks, and reports success. A ZenPlus appliance must never be offered that release. Uploading a package or seeing a row in the release portal is not sufficient acceptance evidence.

Document navigation

- Sections 2 to 4 define the architecture, product isolation, and API contract.
- Sections 5 to 8 define package construction, verification, appliance execution, and recovery.
- Sections 9 to 11 provide the recurring release procedure, rollout policy, and support runbook.
- Sections 12 to 14 provide acceptance tests, delivery ownership, and source references.

2 Architecture and product identity

The release host builds and signs immutable artifacts. The central OTA service authenticates publishers and appliances, stores release metadata, decides eligibility, serves downloads, and records results. The updater on each appliance checks for offers, verifies the artifact, executes the recipe, and reports its result. The application dashboard exposes status and authorized update controls.

Release flow: reviewed source -> build -> offline verification -> publish -> verify catalog -> isolated canary -> controlled expansion -> full rollout -> operational monitoring.

Appliance flow: register -> scheduled check-in -> eligibility decision -> download -> verify -> preflight -> backup -> apply -> schema and functional checks -> commit version -> report.

Item	ZenPlus reference	ZenShield implementation convention
Display name	ZenPlus	ZenShield
Management page	/ota/releases/	/ota/zenai/releases/
Installation directory	/opt/zenplus	/opt/zenshield
Release account	Repository and signing-key owner	zenshield or a dedicated release account
Version marker	/opt/zenplus/.version	/opt/zenshield/.version
Build output	/tmp/zenplus-releases	/tmp/zenshield-releases
Updater units	zenplus-updater.service and .timer	zenshield-updater.service and .timer
Public verification key	zentryc-release.pub	zenshield-release.pub
Signing key	Separate private release key	Independent ZenShield Ed25519 key
API origin	https://zentryc.com	Platform-confirmed API origin and routes

The proposed `zenshield` filesystem name and the existing `zenai` portal path serve different purposes. Do not infer a backend product ID from either string. Store the platform-issued product identity in registration, release records, and authorization rules.

3 Platform setup and isolation contract

The platform team must complete this contract before the first connected ZenShield publish test. The shared ZenPlus publisher currently sends no product field in its release-create payload, and its rollout helper selects a release by version from the list response. Copying those calls without product scoping risks selecting or publishing the wrong product.

Record the following in ZenShield configuration and the platform integration specification:

1. The canonical backend product ID and its mapping to the `zenai` management page.
2. Exact registration, check-in, offer, download, report, release-create, release-detail, publish, rollout, and rollout-pause routes.
3. How product identity is bound to publisher permissions, registration tokens, appliance API keys, release metadata, signing keys, and download access.
4. Supported OS and CPU targets, license rules, release channels, stable group membership, and the dedicated canary appliance IDs.
5. Whether an unrolled published release can be offered. Require no general-fleet exposure until a rollout explicitly enables it.
6. The server's rollout percentage denominator, stable appliance selection rules, promotion semantics, and exact supported pause or withdrawal action.
7. Artifact size limits, upload timeouts, retention, immutable version policy, and response schemas with example success and failure responses.

Isolation requirements: enforce product scope on the server for every read and write, including package downloads and update reports. Filtering the portal or changing a directory name is insufficient. A release ID

from another product must be rejected even if the caller knows it. Use product plus immutable release ID for promotion; do not resolve a release globally by version alone.

If the server uses shared routes, add the platform-defined product selector and enforce it against authenticated identity. If it uses separate routes, configure each route explicitly. Setting the ZenPlus server variable to <https://zentryc.com/ota/zenai/releases/> will not implement either scheme.

Keep ZenPlus and ZenShield signing keys, test appliances, license tokens, storage prefixes, and publisher permissions separate. Demonstrate isolation in both directions before any customer rollout.

4 API integration reference

The following methods and paths are present in the ZenPlus client source. They are the reference contract to adapt, not a claim that ZenShield already exposes the same endpoints. The platform team must supply the ZenShield route mapping and product selector.

Operation	ZenPlus method and path	Authentication and key data
Admin login	POST /api/v1/admin/auth/login	Admin email and password; returns access token
Create release	POST /api/v1/admin/releases/create	Admin bearer token; multipart package and metadata
Publish release	POST /api/v1/admin/releases/{id}/publish	Admin bearer token; immutable release ID
Verify release	GET /api/v1/admin/releases/{id}	Admin bearer token; confirm version, hash, published state
List releases	GET /api/v1/admin/releases	Admin bearer token; enforce product scope and pagination
Create rollout	POST /api/v1/admin/rollouts	Admin bearer token; release ID, stage, group, percentage
Register appliance	POST /api/v1/appliances/register	Registration token and host inventory
Check in	POST /api/v1/appliances/checkin	Appliance bearer key and X-Appliance-ID
Check next offer	GET /api/v1/updates/check	Appliance identity; current_version and arch query parameters
Report result	POST /api/v1/updates/report	Appliance identity and release outcome

Release creation and confirmation

The ZenPlus create request uploads multipart file with version, changelog, severity, package_sha256, and manifest_sig. manifest_sig is base64 of the raw signature embedded in the archive. It optionally sends min_version. ZenShield must also provide or derive the platform-defined product and compatibility scope. Supported ZenPlus severity values are normal, security, critical, and optional.

Creation returns id or release_id; publishing uses that ID. After publishing, read the authenticated detail endpoint and compare ID, is_published, version, and the full package SHA256 with the local artifact. The current publisher does this independently; a successful upload or list response alone is insufficient. ZenShield must additionally verify the returned product identity and target compatibility.

Registration and offers

The reference registration request contains `hostname`, `arch`, `os_version`, `current_version`, and `registration_token`. The response supplies `appliance_id` and `api_key`, with optional subscription data. Persist the issued identity locally with restrictive permissions; never bake an already registered identity into a distributable appliance image.

A check-in offers an update through `next_action`: "update" and a release object. The consumer needs release identity, version, package URL, package SHA256, and any minimum-version constraint; release notes and severity support the UI. Download the returned package URL rather than constructing a file URL from the management page. Require HTTPS and restrict redirects and credential forwarding to trusted download origins.

The separate GET update-check response uses `available`: true with a release object. Preserve the distinction between that response and the check-in response when adapting the client.

Reporting and failure semantics

The reference report body contains `release_id`, `status`, `from_version`, `to_version`, `error_message`, and `log_data`. Current emitted states include downloading, applying, failed, and success. Agree any additional states with the platform before emitting them. Capture rollback outcome separately from the failed update outcome.

Write local history before reporting remotely. Add a durable retry queue and idempotent event IDs in the ZenShield implementation so an offline report is eventually delivered. The current ZenPlus report function logs HTTP failure but does not implement that queue. Redact credentials, customer content, and sensitive configuration from uploaded logs.

5 Repository adaptation and build host

Copy the mechanism into the ZenShield repository and replace product assumptions deliberately. Environment variable overrides alone cannot adapt the current code: paths, service names, database names, health checks, and artifact expectations are also embedded in Python and shell files.

Source component	ZenShield work
<code>scripts/build-release.py</code>	Replace payload lists, build targets, manifest recipe, API scoping, credential location, and artifact checks
<code>scripts/release.sh</code>	Set ZenShield paths and Python; retain clean-source and version checks; make rollout explicitly controlled
<code>scripts/verify-ota-release.py</code>	Replace ZenPlus-specific source and MSI checks; retain signatures, hashes, secret checks, and archive validation
<code>scripts/check-ota-offer.py</code>	Adapt routes, identity, eligibility, and expected product assertions
<code>updater/agent.py</code> and <code>config.py</code>	Replace install, state, and temporary paths; add ZenShield API mapping and policy enforcement
<code>updater/steps</code> and <code>rollback.py</code>	Replace services, databases, runtime commands, backup coverage, and restore procedures
<code>updater/code_inventory.py</code>	Define managed roots and preserved state; rename the payload inventory
<code>updater/systemd</code> and <code>polkit</code>	Rename units and access rules; scope privileged commands to ZenShield

Source component	ZenShield work
scripts/setup-updater.sh	Install public key, configuration, permissions, dependencies, and renamed units
Migration and schema scripts	Port the complete migration ledger and schema gate to the actual ZenShield datastore engines
Local update API and dashboard	Port status, history, check and apply controls with authentication and authorization

Search the copied files for `zenplus`, `netmon`, `/opt/`, database names, unit names, API paths, and hard-coded artifact filenames. Review every match rather than blindly replacing text. Do not copy ZenPlus agents, sensors, pollers, support hooks, or ingestion services unless ZenShield actually requires them.

Provision a release host with the application's pinned toolchain and dependencies. The current wrapper defaults to `/opt/zenplus/venv/bin/python`; older documentation sometimes uses `server/venv`. Select and test one ZenShield runtime path, proposed as `/opt/zenshield/venv/bin/python`. Include cryptography for signing and `httpx` for publishing. Build as the repository and key owner; the current wrapper rejects root execution.

The ZenPlus builder reads `ZENPLUS_DIR`, `ZENPLUS_RELEASE_DIR`, `ZENPLUS_RELEASE_PRIVATE_KEY`, `ZENPLUS_RELEASE_PUBLIC_KEY`, and `ZENPLUS_RELEASE_SERVER_URL`. The wrapper also reads `ZENPLUS_RELEASE_PYTHON` and rollout settings. Rename these consistently in the ZenShield fork or document a temporary compatibility mapping. The command examples in section 9 assume the completed ZenShield adaptation and retain the existing CLI verbs and flags.

6 Package format and release trust

Use one immutable `update-X.Y.Z.zup` file, a gzip-compressed tar archive. Build once, verify once, and promote that exact artifact through all release stages. Preserve its SHA256, source commit, dependency inventory, build logs, and test evidence. A rebuild requires a new release version.

Required package contents

```
update-X.Y.Z.zup
manifest.json
manifest.json.sig
checksums.sha256
code/
  .version
  application source and runtime scripts
  full migration set and migration lockfile
  ZenShield managed code inventory
prebuilt UI and runtime artifacts as required
explicit migration payloads only when needed
```

The reference manifest has `format_version: 2`, `update_id`, `version`, `from_version`, `min_version`, `release_date`, `changelog`, `severity`, `arch`, `os_min`, `steps`, and `rollback_steps`. ZenPlus currently emits `amd64` and `ubuntu-22.04`; replace these with the supported ZenShield target. The reference also includes optional agent metadata that must be adapted or removed consistently with the verifier.

ZenShield contract additions: cryptographically bind product identity and every executable payload to the signed manifest. For example, add a product ID and the SHA256 of a complete payload checksum inventory to the signed manifest. Require matching builder, server validator, and appliance verifier support; version the contract or introduce a tested bridge update when necessary. These additions are requirements for the new implementation, not fields already emitted by the ZenPlus builder.

The current builder signs the exact `manifest.json` bytes and writes `checksums.sha256` separately. A valid manifest signature alone does not authenticate the separate checksum file. The downloaded archive hash currently comes from authenticated OTA metadata. Preserve that whole-package check and add the signed payload binding above. Avoid a circular scheme that tries to embed the final archive's own hash inside that archive.

Verification before any update step

1. Validate the offer's product, release ID, version direction, minimum source version, architecture, and OS eligibility.
2. Download over verified TLS, enforce size and time limits, and check the complete archive SHA256 against the authenticated offer.
3. Reject absolute or escaping archive paths, symbolic and hard links, duplicate entries, special files, and excessive expansion before extraction. Use a private staging directory.
4. Verify the raw Ed25519 manifest signature with the locally installed ZenShield public key. Require a supported manifest format and product identity.
5. Require a complete checksum inventory and verify its signed binding and each payload hash. Reject missing or unexpected executable content.
6. Require agreement between signed metadata, offer metadata, and `code/.version`. Enforce the minimum-version constraint locally as well as on the server.
7. Enforce manifest date policy and valid system time. The reference allows 30 days of age and rejects dates more than 24 hours in the future. Decide a policy that accommodates offline appliances without disabling signature checks.

No private key, `.env`, admin credential file, appliance API key, subscription state, logs, backups, customer data, local certificates, or build cache belongs in the release. Use an allowlist of payload roots plus explicit exclusions and a verifier that fails closed. Preserve appliance-local state during both code replacement and rollback.

Key provisioning and rotation

Generate a separate ZenShield Ed25519 pair on the controlled release host:

```
umask 077
openssl genpkey -algorithm ed25519 -out zenshield-release.key
openssl pkey -in zenshield-release.key -pubout \
    -out zenshield-release.pub
```

Store the private key outside the source tree, readable only by the release account, with a protected off-host recovery copy. Install only the public key in appliance images and register the matching verification key with the platform. The builder must stop when the key is unavailable; the current ZenPlus builder already fails in that case despite older runbook text describing an unsigned warning.

For planned rotation, ship trust in the replacement key through a release verified by the existing key, confirm fleet adoption, then switch signing keys and retire the old trust according to policy. A compromised-key incident needs a separately authenticated recovery process; signing a replacement with a compromised key alone does not restore trust.

7 Appliance update execution

Install a product-specific systemd service and timer. The reference service runs privileged operations as root, from the product directory, with a 30-minute timeout. Use root-owned service definitions and code and a restricted invocation path. The build host release account and appliance updater privilege are separate concerns.

The reference timer starts five minutes after boot and four hours after each activation, with up to five minutes of random delay. The configuration's 900-second interval is not the systemd timer cadence. Carry this schedule into ZenShield initially unless operations chooses another schedule and tests it.

Persist the configuration at `/opt/zenshield/updater/config/agent.conf`, mode 0600, and place the public key at `/opt/zenshield/updater/keys/zenshield-release.pub`. Use separate log, backup, lock, and temporary paths. Require outbound access to the OTA service, trusted download hosts, and any dependency repositories still needed during apply.

Ordered transaction

1. Acquire the product-specific update lock. Check policy, available disk space, compatibility, backup destination, dependency availability, and remaining maintenance time before stopping services.
2. Verify the entire staged artifact as described in section 6. Record the old version and intended release in persistent transaction state.
3. Quiesce ZenShield writes and stop only the services that the release must replace. Preserve the controls needed for recovery.
4. Back up every managed code tree, UI assets, binaries, affected configuration, and every datastore the release can mutate. Abort before mutation if a required backup fails.
5. Apply the new code snapshot. Remove stale files only inside explicitly managed roots while preserving local data, configuration, identity, keys, logs, and backups.
6. Install the required runtime dependencies, prebuilt assets, binaries, and service definitions. Run reviewed, bounded, idempotent hooks. Reload systemd when units change.
7. Run missing migrations in ledger order and verify actual schema state. Fail on unresolved drift even if the migration ledger claims success.
8. Start the intended services and check readiness, schema correctness, UI loading, and a product-specific functional operation such as the relevant event or policy processing path.
9. Only after all gates pass, commit the installed version and transaction success. Report success locally and remotely; retain the required recovery assets.
10. On any apply or validation failure, execute recovery, verify the restored application, and report the original failure and recovery outcome separately.

The reference recipe uses `stop_services`, `backup`, `apt_install`, `apply_code`, `run_hook`, `pip_install`, `run_migration`, `build_dashboard`, `install_binary`, `install_systemd`, `install_config`, `start_services`, and `health_check` handlers. Use only those needed by ZenShield, and reject unknown step types. Configure real ZenShield service names and a real readiness endpoint before generating production manifests.

Policy and restart behavior

The reference configuration declares `auto_update` and maintenance window fields, but the inspected agent execution path does not enforce those fields. Implement and test enforcement in ZenShield before exposing those controls. Explicit manual application and background application should have documented authorization and maintenance semantics.

The current agent can recheck after a successful update and follow an intermediate release chain. Bound that behavior by maintenance time, a maximum hop count, and compatibility gates. Do not treat the first successful intermediate version as final completion if another required update remains.

Add persistent phase tracking and a recovery procedure for reboot or power loss. A process lock prevents simultaneous runs but does not make file replacement or database migration crash-safe. Test interruptions during each mutating phase and ensure an incomplete transaction cannot be reported as successful on restart.

8 Migrations and recovery coverage

Ship the complete migration set on every release so appliances that skip versions can catch up. Keep released migrations immutable and append new migrations to a checksum lockfile. In the reference, lockfile line order is authoritative; filename sorting is not a safe migration order. Commit a new migration and its lock entry together.

The migration runner must maintain a ledger per datastore, apply missing work in order, and support replay-safe operations where reconciliation can rerun a migration. A schema gate must inspect the database itself. ZenPlus emits a schema hook in the release recipe and runs another gate before the final version write, which avoids trusting a healthy HTTP endpoint as proof of a correct schema.

Prefer additive schema changes and an expand-then-contract release sequence. A destructive migration requires an explicit restore or forward-repair plan tested with representative data. Retain bridge releases when direct upgrade from an older version is unsupported, and ensure age policy does not make the bridge unusable.

Recovery scope: the inspected ZenPlus backup code archives managed code and performs a PostgreSQL dump and restore. It does not establish complete ClickHouse recovery or reverse all package-manager, Python environment, external configuration, and systemd changes. ZenShield must inventory each mutation and implement a matching recovery action or a tested forward-recovery strategy.

Keep recovery tied to the current transaction's verified backup. If backup creation fails, restart the original application without restoring an unrelated older backup. Verify dump exit status and usable output; a created filename is not proof of a recoverable database.

After recovery, verify the old code inventory, version, schema compatibility, credentials, and functional health. The reference executor logs rollback-step failures and continues, so its completion log is not proof that restoration succeeded. ZenShield should persist a distinct recovery-failed condition that support can act on.

For a release that already completed successfully, pausing its rollout prevents further offers but does not revert updated appliances. Prefer a new higher-version corrective release. A downgrade needs a separately tested compatibility and data-restoration procedure. Do not overwrite the failed version's published package.

9 Recurring release procedure

The commands below define the intended operator interface after the ZenShield fork and platform integration are complete. Run them from the controlled ZenShield release host as its release account. They are not commands to run against an unmodified ZenPlus checkout. Example versions and release notes must be replaced with the actual release record.

Prepare and build

1. Select the reviewed release commit on the protected release branch. Require a clean working tree synchronized with its remote branch, passing CI, and the intended version in `.version`.
2. Record version, commit, supported source versions, architecture, OS, schema changes, dependency changes, expected downtime, release notes, and recovery plan.
3. Check signing-key access, public-key match, credentials, free space, build dependencies, and platform product scope. Keep secrets outside the repository and logs.
4. Register new migrations during development, review the resulting lockfile changes, and commit them. The release job only lints the committed ledger.

```
cd /opt/zenshield
PYTHON=/opt/zenshield/venv/bin/python
VERSION=1.0.1
MIN_VERSION=1.0.0
PACKAGE=/tmp/zenshield-releases/update- $\{VERSION\}$ .zup
PUBLIC_KEY=/opt/zenshield/updater/keys/zenshield-release.pub

"$PYTHON" scripts/build-release.py lint-migrations
"$PYTHON" scripts/build-release.py build \
  --version "$VERSION" --min-version "$MIN_VERSION" \
  --changelog "Describe the approved changes" \
  --severity normal --skip-agent-artifacts
"$PYTHON" scripts/verify-ota-release.py \
  "$PACKAGE" "$PUBLIC_KEY" \
  --version "$VERSION" --appliance-only
sha256sum "$PACKAGE"
```

This example deliberately uses the reference appliance-only scope. If ZenShield distributes endpoint installers, define and verify its own bundled artifact schema instead of copying the ZenPlus MSI filename or size expectation. Do not skip UI or runtime builds when those components changed. Do not reuse stale local build output.

Test and publish the exact artifact

Install the verified candidate on disposable prior-version appliances and complete the mandatory acceptance tests in section 12. Retain the tested artifact without rebuilding it. Publish only after product scoping and no-rollout exposure behavior have been proven in the integration environment.

```
"$PYTHON" scripts/build-release.py publish \
  --file "$PACKAGE" --version "$VERSION" \
  --min-version "$MIN_VERSION" \
  --changelog "Describe the approved changes" \
  --severity normal --skip-agent-artifacts
```

The command omits a rollout flag. In the ZenShield implementation, this must leave general-fleet eligibility disabled. Capture the returned release ID and independently verify product, ID, version, publication state, architecture, minimum version, signature acceptance, and package SHA256. Check that the release appears under the ZenShield management page and is absent from the ZenPlus catalog.

If upload or publish times out, read the product-scoped catalog before retrying. The server may have committed the operation despite a lost response. Reuse the known release ID where the API permits; never

automatically upload different bytes under the same version.

Canary and promotion

Create a rollout for the exact release ID and dedicated canary group, with automatic promotion disabled. This is a required adaptation: the current ZenPlus rollout helper sends `auto_promote: true` for canary and percentage stages, `promote_after: "24:00:00"`, and `max_failure_pct: 5`. Its CLI does not expose an auto-promotion disable flag.

Use the platform-confirmed API or portal control to create and adjust this rollout. Record its ID, product, membership, percentage, and automatic-promotion state after each change. Avoid creating overlapping active rollouts; establish how the server handles replacement or amendment before using the command-line helper repeatedly.

Trigger a normal check-in on a genuine prior-version canary. Confirm the offer and downloaded hash, allow the appliance to apply it, then verify application behavior, persisted data, local history, installed version, and the server's success report. Continue through the stages in section 10 only when their exit gates pass.

Close the release

Retain the package, signature, public-key fingerprint, source commit, dependency inventory, catalog verification, test evidence, rollout decisions, and support notes. Record fleet success, failure, pending, and offline counts separately. Assign follow-up ownership for failed and offline appliances. Keep published artifacts and required bridge versions for the supported upgrade window.

10 Rollout and release policy

The following is a proposed starting policy for ZenShield. Adopt it in the team's release process or replace it with explicit measured thresholds. It is not a statement of current server enforcement.

Stage	Target	Minimum exit gate
Integration	Disposable prior-version appliances	All mandatory tests pass; product isolation verified
Canary	Named internal ZenShield appliances	Every selected canary reports success and passes functional checks; observe for 24 hours
Limited	10 percent of the eligible stable group	Observe 24 hours; no unresolved release-caused failure or critical regression
Expanded	50 percent of the eligible stable group	Observe 24 hours; health and support signals remain within agreed limits
Full	100 percent of the eligible stable group	Release owner records promotion; monitor remaining failures and offline systems

Use enough canaries to cover supported platforms and upgrade paths. A percentage is meaningful only with a known denominator; for small fleets use explicit appliance membership. Measure offer-to-completion time, application and ingestion health, schema drift, rollback attempts, recovery failures, and support incidents.

Pause immediately on wrong-product offers, signature or payload integrity problems, data loss, schema corruption, recovery failure, or a confirmed critical functional regression. Investigate any canary failure before expansion. The reference helper's 5 percent threshold is a submitted configuration value; prove that the server enforces it before relying on it.

A security or critical release may use an expedited observation window approved by the release owner, with the reason recorded. Severity labels must not bypass signature, compatibility, backup, schema, or health gates. Keep emergency publisher and signing access controlled and audited.

11 Appliance operations and incident handling

The following commands assume the proposed ZenShield directory and systemd names have been implemented. The read-only check still contacts the OTA service and may refresh local subscription information. Starting the updater service can install an offered update.

```
# Inspect installed version and timer
head -n 1 /opt/zenshield/.version
systemctl status zenshield-updater.timer
systemctl list-timers zenshield-updater.timer

# Check an offer without applying it
cd /opt/zenshield
sudo /opt/zenshield/venv/bin/python -m updater --check

# Apply through the normal service during the chosen window
sudo systemctl start zenshield-updater.service
journalctl -u zenshield-updater.service -n 200 --no-pager

# Stop future scheduled checks during incident response
sudo systemctl stop zenshield-updater.timer
```

Stopping the timer does not interrupt an already running update. Do not kill a running migration or restore process casually. Pause the product rollout centrally to stop additional offers, inspect the current transaction phase, and follow the tested recovery procedure.

Symptom	Investigation and action
No release offered	Check registration product, license eligibility, current and minimum versions, OS, architecture, group membership, rollout state, and timer
Signing or verification fails	Confirm the key pair and public-key fingerprint; inspect manifest bytes and artifact hash; never disable verification
Download fails or hashes differ	Check proxy, trusted redirects, size limits, and resume behavior; discard corrupt partial data and retry a verified artifact
Migration or schema gate fails	Stop expansion; preserve logs and backups; recover or issue a corrected migration in a new release
Version advanced but function fails	Treat as release failure; pause offers and investigate readiness coverage and post-commit checks
Recovery fails	Mark the appliance as requiring intervention; preserve transaction data and use the tested datastore recovery procedure
Remote result missing	Inspect local history and retry queue; distinguish report transport failure from installation failure
Older appliance cannot upgrade	Verify minimum-version chain, availability of bridge releases, clock, and manifest age policy
Update exceeds service timeout	Inspect phase and dependency delays; size the timeout from measured worst-case upgrade and recovery duration

Keep customer identity and secrets out of support bundles. Capture appliance ID, source and target versions, release and rollout IDs, transaction phase, failed step, hashes, timestamps, schema diagnostics, and recovery outcome. Never include private signing keys, publisher passwords, or bearer tokens.

12 Acceptance test matrix

Run these tests on the real ZenShield stack. Store evidence with the release record and rerun affected cases when updater, package, platform, or datastore behavior changes.

Test	Required result
Previous version to candidate	Version, managed files, schema, UI, and a representative function pass; local and remote success agree
Oldest supported version and skipped releases	Full migration set catches up or server offers the tested bridge chain
Below minimum version	Client and server block the unsupported jump without mutation
Same version or downgrade offer	No automatic reinstallation or downgrade; explicit recovery procedure remains separate
ZenPlus and ZenShield isolation	Neither product can list, receive, download, promote, or report against the other's protected release
Wrong key and tampered payload	Failure before any service stop or payload execution
Missing or altered checksum inventory	Artifact is rejected, including altered executable bytes with a rewritten unsigned checksum list
Malicious archive entries	Escaping paths, links, duplicates, special files, and excessive expansion are rejected
Local-state preservation	Credentials, license, data, logs, keys, and local settings survive upgrade and recovery
Removed source file	Deleted managed code is absent after upgrade and correct after recovery; unrelated state remains
Backup or disk-space failure	No partial mutation; original application remains usable or is restarted safely
Migration and health failure	Candidate is not reported successful; restoration or forward recovery is verified
Reboot during mutation	Persistent transaction state leads to safe continuation or recovery with an accurate status
Concurrent manual and timer runs	Exactly one update transaction acquires the lock
Auto-update and maintenance policy	Disabled automatic updates and out-of-window checks do not apply a release
Interrupted and resumed download	Range handling works for 206, ignored Range with 200, and 416; complete hash always matches
Offline report and reconnect	One logical result reaches the server through durable retries without false fleet success

Test	Required result
Old or future manifest	Date policy and clock checks behave as documented, including offline bridge upgrades
Canary pause and promotion	Eligibility matches the recorded group and percentage; auto-promotion is disabled as intended
Key rotation	Existing appliances receive new trust safely; old and offline cohorts follow the recovery policy

The current downloader resumes partial files but does not explicitly reset append mode when a server ignores Range and returns 200. Fix and test this behavior in the ZenShield fork. Similarly, do not assume that declaring a configuration option, hash field, or rollback step proves that its enforcement or recovery outcome is correct.

13 Delivery ownership and completion checklist

Owner	Deliverables and completion evidence
Platform team	Product ID, route contract, scoped permissions, storage and signing setup, isolated catalog and rollout behavior
ZenShield engineering	Adapted builder, verifier, updater, managed-state inventory, migration gate, recovery, and local update controls
Release engineering	Controlled build environment, signing-key handling, immutable artifacts, build and publish job, retained release records
QA	Prior-version appliance fixtures, acceptance matrix evidence, failure injection, restored-data and functional checks
Operations and support	Fleet monitoring, pause procedure, recoverability evidence, support diagnostics, and incident ownership
Release owner	Recorded canary and expansion decisions, release notes, customer timing, and release closure

Implementation milestones

1. **Contract complete:** platform supplies the product mapping and exact endpoints; engineering records the actual ZenShield stack, services, state paths, and recovery coverage.
2. **Offline artifact complete:** a signed package builds reproducibly from reviewed source and the adapted verifier rejects invalid and secret-bearing payloads.
3. **Local upgrade complete:** disposable prior-version appliances pass upgrade, migration, functional, and failure-recovery tests.
4. **Connected cycle complete:** registration, offer, download, update, and reporting pass through the platform with bidirectional product isolation.
5. **Controlled rollout complete:** canary selection, pause, manual promotion, and result aggregation work with retained evidence.

6. Operational handoff complete: support can diagnose a failed update and execute the documented recovery procedure; the team can repeat the next release without ad hoc steps.

Release record template

For every release, record: product ID; version; release ID; rollout ID; source commit; build identifier; artifact location and SHA256; public-key fingerprint; supported upgrade paths; OS and CPU targets; migration and runtime changes; test evidence; recovery plan; release notes; canary membership; stage timestamps and decisions; success, failure, pending, and offline counts; known issues; and the accountable release owner.

The handoff is complete when the teams have delivered the six milestones and one end-to-end release has passed the acceptance gates. A portal entry alone does not close the implementation work.

14 Reference materials

Reference baseline: ZenPlus checkout at commit 5f3620be90b422fe52d0e462e64a384dce31063d, inspected on 8 September 2026. Repository-relative paths below identify the source files to transfer or consult with this handoff.

- `scripts/build-release.py` and `scripts/release.sh`: current build, signing, offline verification, publish confirmation, and rollout settings.
- `scripts/verify-ota-release.py` and `scripts/check-ota-offer.py`: artifact and offer validation references.
- `updater/agent.py`, `config.py`, `crypto.py`, `downloader.py`, and `executor.py`: registration, update execution, verification, policy fields, and reporting.
- `updater/code_inventory.py`, `rollback.py`, `schema_gate.py`, and `steps/`: managed code reconciliation, recovery scope, and step behavior.
- `updater/systemd/zenplus-updater.service` and `zenplus-updater.timer`: runtime privilege, timeout, and actual schedule.
- `docs/OTA-RELEASE-WORKFLOW.md`, `Documentation/15-RELEASE-RUNBOOK.md`, and `Documentation/18-MIGRATION-RUNNER.md`: architectural background and migration guidance. Use current source when older command examples or defaults differ.
- ZenShield release management: <https://zentryc.com/ota/zenai/releases/>
- ZenPlus release management: <https://zentryc.com/ota/releases/>

Keep this guide alongside the completed ZenShield API contract and the adapted repository. Update it when routes, trust format, supported upgrade paths, recovery scope, or release policy change.